

Recognition Without Recall: Empirical Limits of Biologically-Inspired Memory Consolidation in Transformers

Aditya Tripathi
adityatripathi1503@gmail.com

Dr. Rachit Garg
Assistant Professor, AI & Data Science
S P Jain School of Global Management

Abstract

This work is relevant to researchers in continual learning, memory-augmented transformers, and biologically-inspired machine learning. We implement and empirically evaluate a biologically-faithful memory consolidation architecture for Large Language Models, drawing on the synaptic tagging-and-capture hypothesis, the Complementary Learning Systems framework, and hippocampal-neocortical replay during sleep. Our system, SLEEP, augments pretrained transformers with five interacting components: prediction-error tagging, direct-write key-value (KV) memory, protein-resource-allocation prioritization, generative replay, and LoRA-based consolidation under explicit safety constraints. Across 200 single-exposure synthetic facts, evaluated over five random seeds on each of four models spanning three families (Qwen2.5-7B and 1.5B, Llama-3.1-8B, Mistral-7B), we systematically isolate where the architecture succeeds and fails. Direct-write KV memory injection produces a clear *recognition* signal, but the same memories do not surface during free-form autoregressive *recall*—a transformer analogue of the recognition-recall dissociation in cognitive psychology, and the recall floor reproduces at the same magnitude on every model we test. We then attempt to close the gap with a retrieval-aware warm-up phase that teaches the frozen model to use its memory during generation; across sixteen runs on the four models the warm-up learns its objective and improves recognition, but recall never moves, so we report it as a replicated negative result. Two further findings sharpen the picture. First, internal surprise-based validation confirms consolidations at a ~ 93 –100% false-confirmation rate against a direct recall test, on every model. Second, running continual learning to ten cycles over nine seeded run-pairs shows that the safety machinery’s apparent advantage is unstable: its damage-capping clip produces a preservation plateau whose existence is universal but whose level varies by three orders of magnitude across seeds, so fixed-horizon, single-run comparisons of continual-learning architectures are unreliable. We map five formalization amendments required for the architecture to function at scale and identify the consolidation step’s stability–plasticity tradeoff as the central open problem.

1 Introduction

Large language models exhibit a form of architectural anterograde amnesia: pretrained weights are fixed at deployment, and any new information acquired during inference exists only within the context window, vanishing once the conversation ends. Retrieval-augmented generation (RAG) treats this as plumbing—append a database lookup to every query—rather than as a representational gap to close [Lewis et al., 2020].

Biological memory faces an analogous problem and solves it through a structured pipeline: surprising experiences are flagged at the synapse by prediction-error-driven tags, those tags compete for a limited pool of plasticity-related proteins (PRPs), and the resulting episodic traces are gradually transferred from hippocampus to neocortex through replay during sleep. We review this literature in Section 2 and Section 3; here we note only that the biological result is continuous, selective, single-exposure learning that does not catastrophically overwrite existing knowledge.

We propose SLEEP, a system that implements all five of these mechanisms—tagging, key-value (KV) memory, protein-resource allocation, sleep replay, and dual-weight consolidation—on Qwen2.5-7B, and that

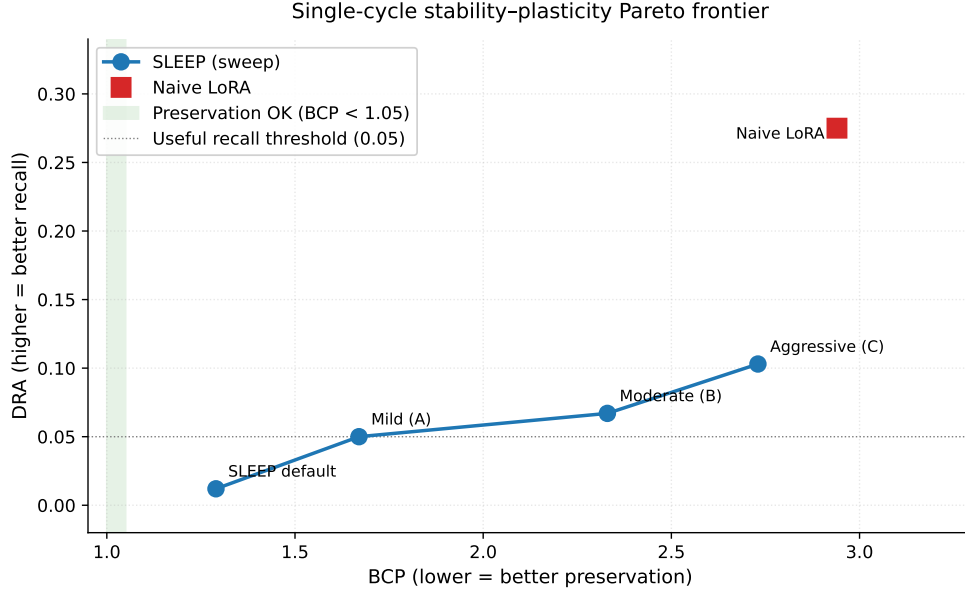


Figure 1: Single-cycle stability-plasticity Pareto frontier. SLEEP configurations sweep from default (most conservative) to aggressive; naive LoRA marks the unconstrained extreme. The green band shows the preservation-acceptable region ($\text{BCP} < 1.05$); the dotted line shows a useful-recall threshold ($\text{DRA} > 0.05$). No setting we tested falls in the upper-left intersection. Section 7.6 shows that this single-cycle picture is itself horizon-dependent.

evaluates each component in isolation. Following a pre-registered formalization in which 36 design questions were resolved before any code was written, we instrument every stage of the pipeline and identify exactly where information is preserved, transformed, or lost. Rather than reporting a single end-to-end metric, this anatomy-of-failure framing localizes the boundary between the mechanisms that transfer to transformers and those that do not. Every headline result is reported across multiple random seeds.

Our results are mixed and specific. The encoding components work: direct-write KV memory injection produces a clear recognition signal on multiple-choice (MC) evaluation. The consolidation components do not: no configuration we tested achieves a Delayed Recall Accuracy (DRA) above 0.05 while holding Base Capability Preservation (BCP) below 1.05, and the recall floor sits at $\text{DRA} \approx 0.006$ on every one of four models across three families, five seeds each. Because a persistent negative result invites the question “can it be fixed?”, we implement and test the fix that the adjacent literature suggests—a retrieval-aware warm-up phase that teaches the model to use stored memory during generation. In sixteen runs across the four models it never moves recall. Finally, running the continual-learning comparison to ten cycles over nine seeded run-pairs shows that the safety machinery’s apparent advantage is an artefact of horizon and seed: the constrained arm’s damage plateau is universal in shape but lottery-like in level, and the cycle-10 ordering against naive fine-tuning flips seed to seed.

These findings yield the following contributions.

Contributions:

- (C1) LoRA fast weights cannot encode at single exposure: $\alpha_{\text{fast}} = 10^{-4}$ falls below the bfloat16 precision floor, and float32 testing shows the default magnitude is separately under-scaled at 7B.
- (C2) Direct-write KV memory injection produces recognition but not generative recall—a transformer analogue of the cognitive recognition-recall dissociation. The recall floor replicates at $\text{DRA} \approx 0.006$ – 0.007 on four models across three families, five seeds each, while the same models follow in-context facts at up to 0.99.
- (C3) Tags are pointers: storing only tagged tokens captures entity mentions without facts; episode-level storage is required.

- (C4) Internal validation passes while external recall fails: across all four models, 93–100% of candidates that pass surprise-reduction fail a direct recall gate—a self-grading discrepancy not unique to SLEEP.
- (C5) No single-cycle Pareto operating point achieves $\text{DRA} > 0.05$ with $\text{BCP} < 1.05$ across our four-setting sweep.
- (C6) Fixed-horizon, single-run comparisons of continual-learning architectures are unreliable: over nine seeded ten-cycle run-pairs, SLEEP’s clip produces a universal step-then-plateau preservation trajectory whose level varies by three orders of magnitude across seeds, flipping the cycle-10 ordering against naive LoRA seed-to-seed—while naive’s recall is higher in 8 of 9 pairs.
- (C7) A retrieval-aware warm-up phase—the treatment the recognition–recall diagnosis implies—learns its training objective and improves recognition, but does not move recall in any of sixteen runs across the four models (≤ 300 steps, general corpus).

We do not claim SLEEP works as a deployable system. We claim something more specific: we show which biological mechanisms transfer to transformers (tagging, attention-based recognition, resource allocation), which do not in the form proposed (generative replay, LoRA consolidation under tight safety), which resist a principled repair attempt, and how strongly the answers depend on evaluation design. We release the implementation, formalization, experimental logs, and the five required amendments.

2 Related Work

We organize related work by methodology rather than chronologically, highlighting how SLEEP relates to each line.

Continual learning via regularization: Kirkpatrick et al. introduced Elastic Weight Consolidation (EWC), which adds a Fisher-information-weighted quadratic penalty to anchor important parameters [Kirkpatrick et al., 2017]. Li and Hoiem proposed Learning Without Forgetting, which uses knowledge-distillation losses against the pre-adaptation model [Li and Hoiem, 2017]. Both require choosing parameters to protect, and both implicitly trade plasticity for stability. SLEEP inherits EWC as one of its safety mechanisms but compounds it with hard clipping, interleaving, and validation rollback; our results in Section 7.5 show that this compound is over-engineered for single-cycle benchmarks but appropriately scaled for multi-cycle dynamics.

Continual learning via replay: Shin et al. proposed deep generative replay, which avoids storing raw old data by training a generator to produce old-task samples [Shin et al., 2017]. Rebuffi et al. introduced iCaRL, which maintains a small exemplar buffer and trains with rehearsal [Rebuffi et al., 2017]. Chaudhry et al. showed that even very small replay buffers suffice for many continual-learning settings [Chaudhry et al., 2019]. SLEEP’s sleep cycle is conceptually closest to generative replay—a compressed sample of the experience drives consolidation—but our results in Section 7.3 and Section 7.4 reveal a specific failure mode: in transformer language models, the generative step does not faithfully reproduce source content even when the source is available in attention via a memory bank, because the autoregressive prior dominates sampling.

Memory-augmented transformers: Wu et al. proposed Memorizing Transformers, which augment standard self-attention with a k -nearest-neighbour lookup over a database of past key-value (KV) pairs and fine-tune the model to use that bank during prediction [Wu et al., 2022]. Our KV memory bank borrows the substrate from this line. The crucial difference is that Memorizing Transformers train the model on the memory-augmented objective, whereas we begin from a frozen pretrained model and ask whether the bank can drive learning without retrieval-aware fine-tuning; the recognition-without-recall result in Section 7.2 is, in part, an explanation of why such fine-tuning is necessary, and Section 7.7 reports what a lightweight version of it does and does not recover. Schmidhuber introduced fast-weight programmers, which predate the transformer but express the same idea in a different substrate—memory implemented as outer-product weight updates [Schmidhuber, 1992].

Gated memory against frozen backbones: A recent line keeps the backbone frozen and learns only a small component that governs how memory is used. Wang et al. equip a transformer with a learnable write-gate and neural router for self-updatable memory [Wang et al., 2024], and G-MemLLM pairs a frozen backbone with a gated latent memory bank, reporting a 13.3% accuracy gain on ZsRE for Llama 3.1 [G-MemLLM Authors, 2026]. Fountas et al. segment the KV cache by *surprise* for infinite-context retrieval—a mechanism arrived at independently of, and closely resembling, our tagging layer [Fountas et al., 2025]. These results motivate the memory gate of Section 5.6: they establish that a small trainable component can, in principle, teach a frozen model to use external memory. Our contribution to this line is a negative one—we report the scale at which that principle fails to transfer into free-form recall inside a consolidation pipeline.

Retrieval-augmented generation: Lewis et al. attach a learned retriever and an external corpus to a generator at inference time [Lewis et al., 2020]. RAG and SLEEP differ in design intent: RAG keeps memory external and re-queries it for every generation, whereas SLEEP attempts to internalize memory into weights so that the model becomes the database. Our results suggest that retrieval-from-attention (the recognition signal) is achievable without retraining but retrieval-into-generation (free-form recall) is not, so the SLEEP–RAG distinction collapses partly: the systems we study still require an attention-side mechanism to make encoded knowledge accessible during generation.

Biologically-inspired machine learning: McClelland et al. proposed Complementary Learning Systems (CLS) theory, which has motivated dual-system architectures in machine learning for two decades [McClelland et al., 1995]; most implementations, however, stop at the conceptual level—two networks with different learning rates—without instantiating the full biological pipeline of tagging, PRP allocation, and sleep replay. Frey and Morris characterized synaptic tagging and capture (STC) [Frey and Morris, 1997], which Redondo and Morris later extended [Redondo and Morris, 2011]; Wilson and McNaughton, and Diekelmann and Born, characterized hippocampal–neocortical replay during sleep [Wilson and McNaughton, 1994, Diekelmann and Born, 2010]. SLEEP is, to our knowledge, the most faithful instantiation of these mechanisms attempted in a transformer language model. Our results characterize precisely which biological mechanisms transfer (tagging, attention-based recognition, resource allocation) and which fail in the form proposed (generative replay-driven consolidation under tight safety).

3 Background

We summarize the biological and machine-learning foundations on which SLEEP is built.

Synaptic tagging and capture: The STC hypothesis posits that weak synaptic stimulation creates short-lived molecular “tags” that can later capture PRPs synthesized in response to a separate strong stimulus, converting transient potentiation into durable change [Frey and Morris, 1997, Redondo and Morris, 2011]. STC explains how the brain implements selective consolidation: tags mark candidates, PRPs are the limited resource, and competition among tags determines which traces become permanent. Nader et al. showed that previously consolidated traces become temporarily labile when re-accessed, allowing revision [Nader et al., 2000].

Complementary learning systems: CLS theory proposes two memory subsystems with distinct learning rates and storage formats: a fast hippocampal system that stores arbitrary episodic associations through one-shot binding, and a slow neocortical system that gradually accumulates statistical regularities [McClelland et al., 1995]. Sleep replay bridges the two systems through hippocampal reactivation of recent experiences, training the neocortex while interleaving consolidated knowledge to prevent overwriting [Wilson and McNaughton, 1994, Diekelmann and Born, 2010].

Parameter-efficient fine-tuning: Hu et al. introduced LoRA, which factors weight updates as $\Delta W = BA$ with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times d}$, and $r \ll d$, training only A and B while freezing the base model [Hu et al.,

2022]. LoRA enables targeted weight updates without full fine-tuning, making it the natural substrate for the slow consolidation system in our architecture.

Memory-augmented attention: Standard transformer attention computes scores between learned queries and keys derived from the current context window [Vaswani et al., 2017]. As reviewed in Section 2, memory-augmented variants extend this with a lookup over stored KV pairs from past contexts; both Memorizing Transformers and earlier fast-weight programmers rely on retrieval-aware fine-tuning to teach the model to use that memory effectively [Wu et al., 2022, Schmidhuber, 1992]. We adopt the KV-memory substrate but place it within a complete biologically-grounded consolidation pipeline rather than treating it as a standalone retrieval mechanism.

Continual learning mechanisms: Our consolidation step inherits two mechanisms reviewed in Section 2: the Fisher-weighted EWC penalty against catastrophic forgetting [Kirkpatrick et al., 2017], and generative replay, which interleaves synthesized old-task samples with new-task training [Shin et al., 2017, Rebuffi et al., 2017].

4 Problem Statement and Research Questions

The mechanisms reviewed in Section 2 and Section 3 are biological. The open question this paper addresses is whether they can be implemented faithfully inside a frozen pretrained transformer, and which of them survive the transfer. We decompose this into four research questions:

- (RQ1) Can a single inference pass deposit retrievable knowledge into a fast-weight store?
- (RQ2) Does information sufficient for recognition also support free-form autoregressive recall?
- (RQ3) Can a sleep-style consolidation step move that knowledge into permanent weights without degrading base capability?
- (RQ4) Do safety mechanisms that appear over-engineered on a single cycle earn their cost across many cycles?

We resolved 36 such design questions in a pre-registered formalization before any code was written; the five amendments that empirical work forced are documented in Annexure A. The remainder of the paper answers RQ1–RQ4 in turn (Sections 7.1–7.6).

5 The SLEEP Architecture

SLEEP augments a frozen base model W_{slow} with three additions:

- (i) A tagging layer that flags surprising spans during inference.
- (ii) A KV memory bank (W_{fast}) into which tagged spans are written one-shot.
- (iii) A consolidation adapter (W_{cons} , a LoRA module) that is updated during periodic sleep phases via replay-driven gradient descent under safety constraints.

A PRP allocator prioritizes which tags get consolidated. Figure 2 summarizes the system. Throughout, labels of the form $Qx.y$ refer to the correspondingly numbered design question in the pre-registered formalization document (Annexure A).

5.1 Tagging Layer

For each input token t at position i , the layer computes the model’s per-token cross-entropy surprise $e_t = -\log p(x_t \mid x_{<t})$. An adaptive z-score thresholder maintains running mean μ and variance σ^2 via an exponential moving average ($\beta = 0.99$) and flags positions where $(e_t - \mu)/\sigma > \kappa$ (default $\kappa = 1.5$). Adjacent flagged positions within $g = 3$ tokens are merged into spans; spans shorter than 4 tokens are discarded. Each surviving span produces a *tag* carrying a key vector $k = \text{Proj}(\bar{h})$ derived from the mean hidden state, an initial strength $s_0 \propto \tanh(E_{\text{span}}/\alpha)$, the source identifier, and the token range. Tags decay exponentially in time and are reinforced when accessed by similar future inputs (Q1.1–1.6).

SLEEP architecture: wake (left/middle) and sleep (right)

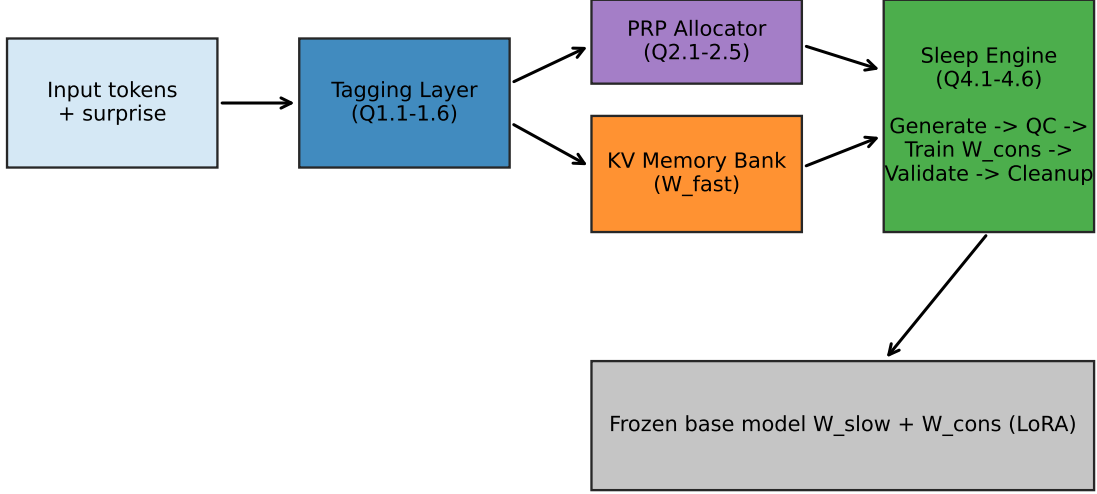


Figure 2: The SLEEP architecture showing wake phase (left) and sleep phase (right). Wake: input tokens are processed by the tagging layer; tags allocate against the PRP budget and write episode-level K/V into the memory bank. Sleep: bank-conditioned generative replay produces synthetic training samples that drive gradient updates to W_{cons} under EWC and hard-clipping safety constraints, after which the bank is cleared.

5.2 KV Memory (W_{fast})

A central design correction in this work is that *tags are pointers, not storage*. The original formalization stored only the K and V projections of the tagged tokens themselves; in practice this captured isolated entity mentions (“Nimbus Holdings”) without their factual contents (revenue figures, dates). We replace this with *episode-level* storage: when a tag fires, we extract pre-RoPE keys and raw values for the full sentence containing the tag from each adapted attention layer. Concretely, for adapted layer ℓ , the memory key and value are computed by two separate projections of the layer-normed episode hidden state:

$$K_{\text{mem}}^{(\ell)} = W_K^{(\ell)} \cdot \text{LayerNorm}_{\ell}(h_{\text{episode}}^{(\ell)}), \quad (1)$$

$$V_{\text{mem}}^{(\ell)} = W_V^{(\ell)} \cdot \text{LayerNorm}_{\ell}(h_{\text{episode}}^{(\ell)}), \quad (2)$$

where $h_{\text{episode}}^{(\ell)}$ is the input hidden state at layer ℓ across the full episode, captured via `output_hidden_states=True`. Crucially, the layer-norm is applied before the projections to match what the model computes during its own attention forward pass; without this correction, extracted K/V differ from in-attention K/V by exactly one normalization, producing silently wrong attention scores.

At inference, stored K/V are injected as a prefix at *negative position IDs* $[-n_{\text{mem}}, -1]$ in each adapted layer’s attention. We apply Rotary Position Embedding (RoPE) to memory keys at read time [Su et al., 2024]; because RoPE is a relative-position rotation, the memory at negative positions appears to current queries as “recent prefix tokens,” a regime the model was trained to handle. Following Wu et al., per-query top- k retrieval gating restricts attention to the k memory positions whose Q·K inner product is highest, implemented through a $-\infty$ additive mask on excluded positions in the model’s existing attention machinery [Wu et al., 2022].

The bank is per-tag indexed and supports per-tag eviction. Capacity is bounded by total stored tokens; the capacity policy is delegated to the PRP allocator.

5.3 PRP Allocation

A composite score combines four signals (Q2.1):

$$\text{score}(\tau) = w_1 \cdot E_{\text{span}}^{\text{norm}} + w_2 \cdot \text{access}^{\text{norm}} + w_3 \cdot \text{xref} + w_4 \cdot e^{-\Delta t / \tau_r}, \quad (3)$$

with weights (0.35, 0.30, 0.15, 0.20). Cross-reference density xref measures how many other tags have similar keys (cosine similarity > 0.5). PRP allocation is competitive: only tags whose score exceeds an adaptive threshold θ_{prp} receive an allocation token $p = 1$, and the budget $B = c_{\text{prp}} \cdot N_{\text{params}}^{(B)}$ caps the total. Allocated tags become candidates for sleep consolidation.

5.4 Sleep Engine

A sleep cycle proceeds in five phases, with KV memory injection enabled or disabled per phase to enforce the intended semantics:

1. **Generate** (KV active). For each candidate tag, produce a replay sample by autoregressively sampling from the model conditioned on the early tokens of the source episode. With KV memory active, the model can attend to the original experience while compressing it.
2. **Quality check**. Replays are retained if (a) cosine similarity to the source > 0.5 and (b) mean per-token surprise exceeds a calibration threshold μ_{surprise} .
3. **Train** (KV disabled). W_{cons} is updated by gradient descent on the accepted replays interleaved 1:1 with self-generated samples drawn from the unadapted base model (the “old knowledge” arm). Training disables KV injection so that W_{cons} learns to encode the replay content into weights rather than relying on the bank as a crutch.
4. **Perplexity check** (optional, KV disabled). On benchmark data, post-training perplexity is compared to pre-training perplexity. If degradation exceeds $\epsilon_{\text{degrade}}$, W_{cons} is rolled back.
5. **Validate & cleanup** (KV disabled). For each candidate tag, the post-training model’s surprise on the original span is compared to the pre-training surprise; the tag passes if the reduction is at least ϵ_{learn} (default 10%). Passing tags are removed from the buffer; failing tags are penalized; tags that fail repeatedly are evicted permanently.

After successful consolidation, the KV bank is cleared—its content has, in principle, transferred to W_{cons} .

5.5 Safety Machinery

W_{cons} training applies four safety mechanisms simultaneously. Each targets a distinct failure mode of replay-driven gradient updates:

- **EWC penalty**: a Fisher-weighted quadratic regularizer pulls W_{cons} toward its pre-training checkpoint, with strength λ_{EWC} (default 100) [Kirkpatrick et al., 2017].
- **Hard clipping**: after each optimizer step, per-parameter changes are clipped to magnitude δ_{max} (default 0.01) relative to the pre-cycle checkpoint, and gradients are scaled by per-layer plasticity coefficients $\phi_{\ell} \in [\phi_{\text{min}}, 1]$ that encode “hippocampal-like” top-third layers as more plastic.
- **Old-knowledge interleaving**: each training step mixes replay samples with self-generated continuations from the unadapted base model, providing regularization pressure toward preserving base capabilities.
- **Validation rollback**: candidates whose surprise on the original span does not drop by $\epsilon_{\text{learn}} = 0.10$ are rolled back to their pre-cycle parameter values.

The cumulative effect is a strong stability bias. The empirical question is whether this bias is correctly calibrated. Section 7 shows it is not, in single-cycle settings; Section 7.6 shows that whether it is justified in multi-cycle settings depends on how many cycles one runs.

5.6 Retrieval-Aware Warm-Up

The five components above constitute SLEEP as originally specified. We add one further component, motivated directly by the empirical finding in Section 7.2: memory that supports recognition does not support generation, because a frozen pretrained model has never been trained to condition its next-token distribution on retrieved K/V. Wu et al. resolve this by training the model on a memory-augmented objective [Wu et al., 2022]; gated-memory architectures reach the same end with a small trainable component against a frozen backbone [G-MemLLM Authors, 2026, Wang et al., 2024]. We adopt the lightest form of this idea that fits inside SLEEP.

Mechanism: Before any continual learning begins, we run a one-time warm-up on a general corpus that contains none of the evaluation facts. For each step we split a passage into a prefix and a continuation, write the prefix’s episode K/V into the memory bank, and train on the continuation with that memory injected. Because the prefix is genuinely informative about the continuation but reaches the model only through memory, lowering the loss requires learning to route memory into generation. The warm-up therefore teaches a *skill*, not a fact.

The trainable component: We introduce a *memory gate*: a per-layer, per-KV-head multiplicative scale $g_h^{(\ell)} > 0$ applied to the memory values before they enter attention,

$$\tilde{V}_{\text{mem}}^{(\ell)} = g^{(\ell)} \odot V_{\text{mem}}^{(\ell)}. \quad (4)$$

Because attention output is linear in V and the softmax weights depend only on $Q \cdot K$, this scales exactly the memory contribution to the residual stream and nothing else. The gate is parameterised as a log-scale initialised at zero, so an untrained gate is exactly the identity and installing it changes no computation. It holds $|\mathcal{L}_{\text{adapted}}| \times n_{\text{kv-heads}}$ parameters—36 for the top third of Qwen2.5-7B. We also test a higher-capacity variant in which the W_{cons} LoRA adapter trains alongside the gate.

Crucially, the gate lives on the attention path rather than inside W_{cons} , so it persists across sleep cycles that overwrite the consolidation adapter. Section 7.7 reports what this buys.

6 Experimental Setup

Models: The primary model is Qwen2.5-7B with bfloat16 weights, eager attention, and no quantization [Qwen Team, 2024]: 28 transformer layers, 28 query heads, 4 key-value heads (grouped-query attention), head dimension 128. To test generality we repeat the core protocol on Qwen2.5-1.5B (same family, weaker prior), Llama-3.1-8B, and Mistral-7B (different families, different pretraining corpora). In every case we adapt the top third of layers for both KV memory injection and the W_{cons} LoRA module; the adapted set is derived from each model’s layer count rather than hardcoded. Before any experiment on a new architecture, a verification harness asserts three ground-truth properties of the injection path on the real model—empty-bank identity, populated-bank effect, and disable-restores-baseline—and the architecture is excluded if any check fails (none were).

Dataset: We construct a controlled benchmark of 200 synthetic facts spanning 10 templates: corporate financials, scientific discoveries, city foundings, regulatory protocols, world records, technology architectures, medical trials, sports records, album releases, and geological events. Each fact is a single sentence containing fictional entity names, specific numerical values, and contextual details. Templates are deliberately structured so that the surprising tokens (entity names like “Nimbus Holdings”) cluster at the start of each fact, with factual content (revenue figures, dates, percentages) following. Each fact is paired with a question prompt and a list of three keywords expected in a correct answer. The dataset is generated by deterministic seeded Python code; see Annexure for the generator.

We deliberately use synthetic facts rather than natural documents to isolate single-exposure encoding from prior-knowledge confounds. Real-world facts may have been seen during pretraining, making it impossible to distinguish “the model recalls because it learned” from “the model recalls because it always knew.”

Evaluation metrics: We measure three forms of recall and one preservation metric:

Multiple-choice accuracy (MC): for each fact, the correct keyword and three same-template distractors are presented as options A–D in a randomized order seeded by fact ID. We compute the model’s logit at each option’s first token following “Answer:” and report the fraction where the argmax matches the correct option. We also log $P(\text{correct})$ and $P(\text{correct} \mid \text{wrong})$ via softmax over the four option-letter logits, providing a calibration view sensitive to partial encoding.

Cloze accuracy: the prefix of the fact text up to (but not including) the first keyword is provided as a prompt; the model generates 20 tokens; we score 1 if the keyword appears in the generation, 0 otherwise.

Free-form delayed recall accuracy (DRA): the fact’s question prompt (e.g., “What was Nimbus Holdings’ Q3 revenue?”) is provided; the model generates 50 tokens with temperature 0.7; we compute the fraction of expected keywords appearing in the generation. The aggregate DRA is the mean across facts.

Base capability preservation (BCP): we measure perplexity on three held-out control texts (capital of France, water boiling temperature, Python’s history) before and after each modification, and report the ratio $\text{BCP} = \text{PPL}_{\text{after}} / \text{PPL}_{\text{before}}$. $\text{BCP} \leq 1.05$ is the preservation threshold (no meaningful degradation); $\text{BCP} > 2$ indicates substantial damage.

Baselines: The **base model** is the unmodified Qwen2.5-7B. **Naive LoRA** is a single LoRA adapter (rank 16, $\alpha = 32$, applied to V/O projections of the same top-third layers as W_{cons}) trained directly on the 200 facts via uniform-with-replacement sampling for 100 gradient steps with AdamW (lr 10^{-4} , weight decay 0.01, batch size 32). This matches SLEEP’s compute budget but applies no safety constraints. We add three reference points that bound the problem from opposite directions: **in-context injection** places the gold fact directly in the prompt with no training at all (an upper bound on what the model can do with a fact it can see); **RAG** retrieves the top-3 facts by mean-pooled embedding similarity and prepends them [Lewis et al., 2020]; and **EWC-only** applies the Fisher-weighted penalty to naive LoRA with none of SLEEP’s other machinery, isolating how much of any preservation benefit is attributable to EWC alone.

Seeds and variance reporting: Every headline result is run across multiple random seeds, each in an independent process so that model initialization, tagging decisions, replay sampling, and MC option ordering all vary. The single-cycle pipeline and its recall/preservation metrics use five seeds (0–4) *per model* and we report mean $\pm$ standard deviation throughout. The warm-up experiments use two seeds per variant per model (sixteen runs total). The ten-cycle continual-learning protocol runs three seeds on Qwen2.5-7B and two on each other model (nine run-pairs); because its seed variance turns out to be the finding rather than noise, Section 7.6 reports per-run trajectories rather than averages.

Warm-up protocol: The retrieval-aware warm-up (Section 5.6) trains on 400 passages drawn from WikiText-2, filtered to 80–400 characters and containing none of the evaluation facts. Each step samples a passage, splits it at a point drawn from the middle two quarters, writes the prefix episode into the bank, and takes one AdamW step (lr 10^{-3} , gradient clipping 1.0) on the continuation with memory injected. We report 300-step runs; a 150-step configuration was also tested and is reported in Section 7.7. After warm-up the gate is frozen and the bank cleared, so evaluation begins from an empty memory exactly as in the no-warm-up condition.

Hyperparameters: All non-overridden values track the formalization defaults. We document four hyperparameter overrides applied during the empirical work, each with its empirical justification logged at the time the change was made (see the logbook entries cited in Annexure A): $\alpha_{\text{slow}} = 10^{-4}$ (precision floor under bfloat16; the 10^{-5} default produces zero detectable parameter movement), $\delta_{\text{max}} = 0.01$ (model-scale-dependent; 10^{-3} saturates within 30 steps on 7B), KV top- $k = 64$ (must scale with episode size; episodes are ~ 30 –40 tokens so $k = 16$ retrieves only fragments), and quality-check μ_{surprise} disabled (the formalization’s absolute-surprise threshold suffers from a generator-discriminator gap on self-generated replays). The full hyperparameter list is given in Table 8 of the Annexure.

Compute: Experiments run on single NVIDIA GPUs via RunPod: an RTX 5090 (32GB) for the initial single-cycle work and RTX A6000s (48GB) for the multi-seed, ten-cycle, warm-up, float32, and cross-model programmes—single precision at 7B and the full four-model matrix both require more than 32GB. Total

Table 1: W_{fast} substrate comparison on the 200-fact wake-only protocol. KV injection produces a recognition signal that LoRA does not; KV without retrieval gating destroys BCP.

	MC Tagged	MC Untagged	Δ (T-U)	BCP
LoRA W_{fast} ($\alpha = 10^{-4}$)	0.23	0.24	-0.01	0.99
LoRA W_{fast} ($\alpha = 10^{-3}$)	0.24	0.24	0.00	1.17
KV memory (no gating)	0.27	0.16	+0.11	96.88
KV memory ($k = 16$, gated)	0.28	0.12	+0.16	1.08

compute for all experiments reported in this paper is approximately 25 GPU-hours, totaling roughly USD \$20 in cloud cost. Each end-to-end run (200 facts, full sleep cycle, complete evaluation suite) takes 8–15 minutes at 7–8B scale. We log every run’s command line, configuration, output, and resulting metrics; these are released alongside the implementation.

Reproducibility: The implementation comprises approximately 3,500 lines of Python across six modules (tagging, prp, weights, sleep_engine, orchestrator, evaluation), with 297 unit and integration tests. The 36-question formalization and the empirical amendments derived from it are released as a separate document to enable independent reproduction. The exact RunPod instance image, Python version, and `transformers` library version (5.7.0) are documented; `attn_implementation=eager` is forced because the Scaled Dot-Product Attention (SDPA) path handles 4D additive masks differently and silently degrades the gating mechanism.

7 Results

The paper’s empirical contribution is an anatomy of where in the SLEEP pipeline information is preserved, transformed, or lost. We organize the results around seven questions, each with a specific experimental answer: where encoding succeeds (§7.1), why it does not become recall (§7.2), what storage granularity is required (§7.3), where consolidation stalls (§7.4), what the safety parameters buy (§7.5), how the answers change with evaluation horizon (§7.6), and whether the recognition–recall gap can be repaired (§7.7).

7.1 Encoding: LoRA Fast Weights vs. KV Memory Injection

Proposed: The original SLEEP design specified LoRA adapters as the fast-weight substrate. We find this configuration cannot deposit retrievable knowledge from a single inference pass, and we show that direct-write KV memory injection produces a clear—if limited—encoding signal.

Setup: We compare two W_{fast} substrates on an identical 200-fact wake-only protocol: (i) LoRA W_{fast} updated by gradient descent on each tagged span (the original specification), and (ii) KV memory injection with episode storage and top- k retrieval gating ($k = 16$). For each, we evaluate multiple-choice accuracy and BCP after wake (no sleep cycle).

Results: Table 1 reports the contrast. LoRA W_{fast} at $\alpha_{\text{fast}} = 10^{-4}$ produces no detectable encoding signal: tagged-fact MC accuracy (0.23) is statistically indistinguishable from untagged-fact accuracy (0.24). Increasing the learning rate to 10^{-3} degrades the model (BCP 1.17) without producing recall. We attribute this to the bfloat16 precision floor: at $\alpha_{\text{fast}} = 10^{-4}$, single-step gradient updates are below the dtype’s representable epsilon for typical weight magnitudes.

KV memory injection with retrieval gating produces a +0.16 multiple-choice gap between tagged and untagged facts, with BCP held at 1.08 (Figure 3).

Calibration analysis is more sensitive: the average probability assigned to the correct option, conditional on the model picking the wrong one, is 0.23 for tagged facts versus 0.19 for untagged—suggesting a partial encoding signal that does not quite flip predictions. Without retrieval gating, the KV bank’s contents drown attention on irrelevant queries, destroying base capability (BCP 96.88).

KV memory injection produces a recognition signal that LoRA does not

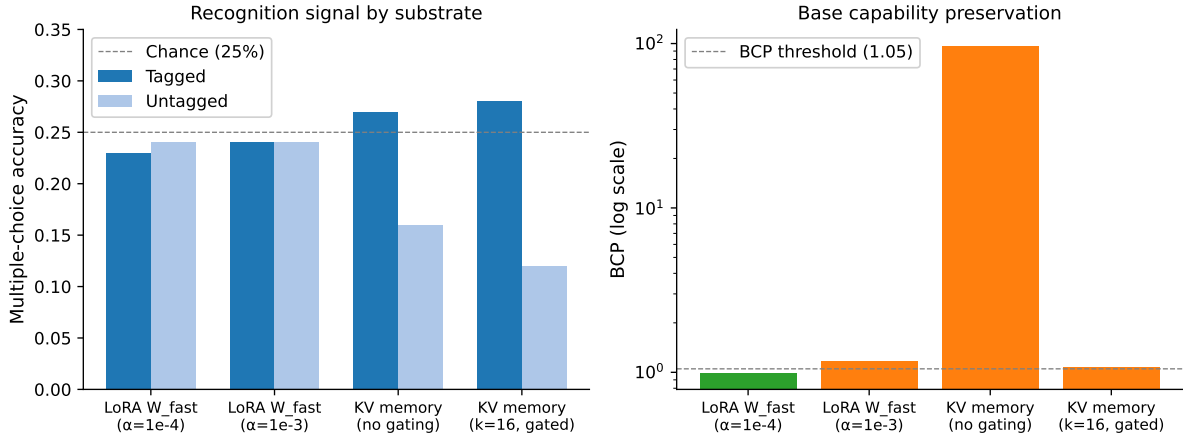


Figure 3: Comparison of W_{fast} substrates on a 200-fact wake-only protocol. Left: tagged-vs-untagged multiple-choice accuracy. Right: BCP (log scale, since the no-gating KV configuration produces $\sim 100\times$ degradation). The KV substrate with retrieval gating is the only configuration that produces a measurable recognition signal (Tagged – Untagged = +0.16) while preserving base capability.

7.2 Recognition Without Recall

The KV memory result above is structurally partial. Tagged-fact *multiple-choice* accuracy exceeds untagged by +0.16, but tagged-fact *cloze* accuracy is 0.00 and tagged-fact *free-form* accuracy is 0.00. The same memories that bias four-way recognition fail to drive autoregressive token generation. We name this dissociation the *recognition–recall gap* and treat it as the paper’s central novel finding: it is not a defect of our specific implementation but, we argue, a structural property of memory-augmented attention over frozen pretrained models.

The cognitive-psychology analogue: The recognition–recall dissociation is a foundational observation in episodic memory research [Morris, 2006]. Recognition (“does this seem familiar?”) can be supported by partial trace reactivation—a sense of fluency or matching—while free recall (“what came next?”) requires reconstructing the trace in sufficient completeness to generate its content. Patients with hippocampal damage often retain residual recognition ability even when free recall is severely impaired. The structural distinction in cognitive psychology is that recognition is a similarity-judgment task (compare the probe to stored representations), while recall is a generation task (produce the stored representation autoregressively from a cue). Our result reproduces this distinction in transformer language models for a different reason at the mechanistic level, but with the same operational consequence.

The mechanistic explanation: At each generation step, attention to memory K/V provides additive perturbations to the pre-softmax logits at every layer. For our $n_{\text{mem}} \approx 5,500$ stored tokens with top- k retrieval gating, the k retained memory positions contribute at most $k/(k + |\text{seq}|)$ of total attention mass, with the remainder going to the current sequence’s tokens whose keys are RoPE-anchored at the model’s normal training distribution. Memory thus enters the next-token prediction as a small perturbation against a strong prior trained on trillions of tokens. The perturbation is sufficient to flip an argmax in a four-way comparison—all four options share the same prior and the memory tilts the relative scores—but is overwhelmed across an autoregressive generation, where each generated token comes from a categorical distribution over a 150,000-token vocabulary in which the prior dominates. A memory entry can nudge the most likely token of a sentence; it cannot consistently steer 50 generated tokens against the model’s natural continuation distribution.

Why this matters beyond SLEEP: Memorizing Transformers, of Wu et al., avoid exactly this failure mode by retraining the model on memory-augmented losses, teaching the next-token distribution to depend on retrieved K/V [Wu et al., 2022]. Our recognition-without-recall result is—in part—a precise specification of *what* that fine-tuning fixes: the gap between attention-time bias and sampling-time use. For applications that need a frozen pretrained backbone with appended memory, the gap is structural and cannot be closed by hyperparameter tuning. We return to the architectural implications in Section 8.

7.3 Episode Storage: Tags Are Pointers

Proposed: The original formalization stored only tagged-token K/V; we show this captures entity mentions without their factual content, and that storing the full sentence containing the tag is necessary for replay quality.

Diagnostic: We inspect the generated replays produced by the sleep cycle when the bank stores only the tagged span. With seeds like “Nimbus Holdings reported Q”, the model generates fluent financial-news text with fabricated revenue figures (“\$32.6 million”, “up 6.1% from prior year”)—values not in the original fact. The pattern persists across templates: stored entity-name K/V provide an attention bias toward “earnings statement” representations without supplying the specific facts.

When storage is expanded to the full sentence containing the tag (the *episode*), and the seed for replay generation is taken from the start of the episode rather than from the tag location, replay coherence improves: the consolidation rate (validation passes / candidates) rises from 40% to 62%, and BCP improves from 1.06 to 1.008 ($k = 16$, episode storage). The recall metrics, however, do not improve correspondingly—a finding we examine in Section 7.4.

This empirical correction validates the proposal’s own framing: *tags are pointers, not storage*. The implementation must store what the pointer indexes, not the pointer’s surface form.

7.4 The Consolidation Bottleneck

Proposed: Even when generative replay is replaced with the original tokens (eliminating replay-quality as a confound), SLEEP’s safety machinery prevents sufficient W_{cons} updates for free-form recall.

Setup: We add a *replay-strategy* parameter to the sleep engine. With `replay_strategy="original"`, the engine bypasses generation entirely and uses the original fact tokens as replays. All other components—PRP allocation, EWC, hard clipping, validation, rollback, KV-disabled training—run unchanged. This isolates “does W_{cons} training learn from data, given perfect data?”

Result: With original-token replays under default safety parameters ($\delta_{\text{max}} = 0.01$, $\lambda_{\text{EWC}} = 100$, $\alpha_{\text{slow}} = 10^{-4}$, ~ 100 steps), recall remains at floor across five seeds: $\text{DRA} = 0.006 \pm 0.003$ with $\text{BCP} = 1.066 \pm 0.032$. Perfect replay data does not rescue consolidation.

Quantifying the self-grading discrepancy: To measure—rather than argue—the mismatch between internal and external criteria, we add a second validation stage. Stage one is the original surprise-reduction check; stage two is a direct free-form recall test that asks whether the post-training model can produce any expected keyword from the fact’s question prompt under greedy decoding. Table 2 reports both gates across five seeds.

The gap is a factor of roughly one hundred: **$\sim 97\%$ of the consolidations the system certifies internally fail an external recall test**, and the effect is consistent on every seed. *Internal validation criteria can pass while external recall fails*. We argue this self-grading discrepancy is a methodological concern broader than SLEEP: a system that grades itself on a proxy of its own behavior provides no protection against proxy–target mismatch, and the magnitude here suggests the mismatch can be near-total rather than marginal.

Table 2: Two-stage validation across five seeds (200 facts, original-token replay). The surprise proxy confirms roughly 39 consolidations per run; fewer than one survives a direct recall test.

Quantity	Mean $\pm$ SD	Per-seed values
Passed surprise-reduction (stage 1)	38.8 ± 0.4	39, 39, 39, 39, 38
Passed direct recall gate (stage 2)	0.4 ± 0.5	0, 1, 1, 0, 0
DRA	0.006 ± 0.003	.005, .007, .010, .003, .005
BCP	1.066 ± 0.032	1.11, 1.03, 1.04, 1.09, 1.06

Table 3: The recall floor and self-grading gap across four models, three families (5 seeds each, mean $\pm$ SD). The floor and the proxy-recall mismatch replicate everywhere.

Model	DRA	BCP	Proxy passes	Recall-gate passes
Qwen2.5-7B	0.006 ± 0.003	1.066 ± 0.032	38.8 ± 0.4	0.4 ± 0.5
Qwen2.5-1.5B	0.007 ± 0.002	0.997 ± 0.003	23.8 ± 1.1	0.0 ± 0.0
Llama-3.1-8B	0.006 ± 0.003	0.958 ± 0.009	9.2 ± 2.4	0.6 ± 0.5
Mistral-7B	0.007 ± 0.000	1.001 ± 0.002	4.0 ± 1.0	0.0 ± 0.0

Cross-family and cross-scale replication: Because both the recall floor and the self-grading gap could in principle be artefacts of one model’s pretraining, we repeat the five-seed protocol on three further models spanning two additional configurations of the same family and two other families. Table 3 shows the outcome: the free-form recall floor sits at $\text{DRA} \approx 0.006\text{--}0.007$ on every model, and the direct recall gate confirms at most a fraction of one consolidation per run everywhere, against 4–39 proxy confirmations. The number of *candidates* varies substantially across families—Mistral’s surprise statistics allocate a tenth as many as Qwen—but the false-confirmation structure is invariant. Notably, the same models follow in-context facts well (Table 5), so the parametric recall floor is not a comprehension failure.

7.5 The Stability–Plasticity Pareto Frontier

Proposed: Sweeping the four safety knobs produces a smooth Pareto curve with no operating point achieving both meaningful recall and preservation on the single-cycle benchmark.

Setup: Using original-token replay (clean signal), we sweep four settings of $(\delta_{\max}, \lambda_{\text{EWC}}, \alpha_{\text{slow}}, \text{steps_per_memory})$ from SLEEP defaults to near-naive-LoRA. We compare against naive LoRA fine-tuning at matched compute. Table 4 reports the curve.

The curve (Figure 1, also shown on page 1 as the paper’s central result) is monotonic: relaxing safety raises DRA and raises BCP simultaneously. There is no knee; no inflection point where DRA jumps without BCP cost. On a single-cycle 200-fact benchmark, the SLEEP architecture’s value proposition—“a small recall sacrifice in exchange for guaranteed preservation”—does not hold: every SLEEP point pays more BCP per percentage point of recall than naive LoRA does at its single endpoint.

Reference points: Table 5 places these numbers against three baselines that bound the problem. Supplying the fact in context yields $\text{DRA} = 0.760$ at no preservation cost, establishing that the model reads and uses a fact it can see; RAG’s shortfall against that ceiling (0.213) is therefore attributable to retrieval rather than to comprehension. Most pointedly, EWC applied alone—without hard clipping, interleaving, validation rollback, tagging, or sleep scheduling—reaches $\text{DRA} = 0.070$ at $\text{BCP} = 1.116$, a better recall-per-preservation ratio than any full SLEEP configuration in Table 4.

Single-cycle interpretation: A natural reading of these single-cycle results is that SLEEP’s safety machinery is over-engineered. The mechanisms (EWC, hard clipping, interleaving, validation rollback) collectively dampen learning more than they protect base capability, and the EWC-only baseline suggests that

Table 4: Single-cycle Pareto frontier on the 200-fact benchmark. No SLEEP setting achieves $\text{DRA} > 0.05$ with $\text{BCP} < 1.05$. Naive LoRA achieves higher DRA at unacceptable BCP cost.

Setting	$\delta_{\max}$	λ_{EWC}	α_{slow}	steps/mem	DRA	BCP
SLEEP default	0.01	100	10^{-4}	5	0.012	1.29
Mild relax (A)	0.02	50	10^{-4}	4	0.050	1.67
Moderate (B)	0.05	10	3×10^{-4}	4	0.067	2.33
Aggressive (C)	0.10	1	5×10^{-4}	6	0.103	2.73
Naive LoRA	∞	0	10^{-4}	—	0.275	2.94

Table 5: Baseline reference points across models (100 facts, single seed each). In-context injection bounds the task from above everywhere; on 7B-class models EWC-only achieves a better recall-per-preservation ratio than any full SLEEP configuration, though the effect weakens at 1.5B.

	Qwen-7B	Qwen-1.5B	Llama-8B	Mistral-7B
In-context DRA	0.760	0.700	0.987	0.927
RAG DRA	0.213	0.460	0.520	0.563
EWC-only DRA (BCP)	0.070 (1.12)	0.033 (1.43)	0.073 (1.47)	0.157 (2.53)

the marginal machinery beyond the Fisher penalty is not paying for itself. On a one-shot benchmark, that is a true description of what they do. Section 7.6 tests whether it remains true as the horizon lengthens.

7.6 Evaluation Horizon and Seed: The Long-Run Comparison Is Not Stable

Proposed: SLEEP’s safety mechanisms are designed for repeated-cycle continual learning rather than single-cycle benchmarks. We therefore evaluate in that regime—and find that the comparison against naive fine-tuning is not stable in the number of cycles, which changes what can be claimed.

Setup: We split 200 facts into 10 disjoint batches of 20. Across ten sequential cycles the model sees one batch per cycle. After each cycle we measure (i) DRA on the cumulative facts seen so far (the forgetting test), (ii) DRA on the current cycle’s facts (the learning test), and (iii) BCP on controls (the preservation test). We compare **SLEEP-A** (Setting A from the Pareto sweep) against **Naive LoRA** (200 steps per cycle on that cycle’s batch). Both arms accumulate weight updates across cycles—each cycle starts from the previous cycle’s adapter state. We run this protocol with three seeds on Qwen2.5-7B and two seeds on each of Qwen2.5-1.5B, Llama-3.1-8B, and Mistral-7B: nine SLEEP runs and nine naive runs in total.

Result—a universal shape with a lottery for its level: Table 6 reports all nine run-pairs. Three regularities hold in every run. First, the *step-then-plateau* signature: each SLEEP run holds BCP near 1 for its first cycles, then jumps discretely and stays flat—within a few percent—for every remaining cycle (9/9 runs). Second, naive LoRA’s cumulative recall exceeds SLEEP’s in 8 of 9 pairs, typically by 2–20×. Third, no run of either arm ever satisfies $\text{BCP} < 1.05$ beyond cycle two. What does *not* replicate is the plateau level: across seeds of the *same configuration* it spans 2.7–10.2 on Qwen-7B, 16–230 on Qwen-1.5B, and 4.8–1531 on Llama—three orders of magnitude on identical hyperparameters. Which arm “wins” preservation at cycle 10 accordingly flips seed to seed (SLEEP wins 4 of 9 pairs, naive 5 of 9).

Interpretation—horizon and variance jointly decide the ordering: Three successively larger experiments gave three different headlines: a single cycle favours naive LoRA; three cycles favour SLEEP (the ordering we ourselves reported from the three-cycle experiment); ten cycles across seeds shows the ordering is not stable at all. The seed-stable claims are structural, not comparative: SLEEP’s clipping mechanism caps damage at a plateau whose *existence* is guaranteed but whose *level* is effectively a lottery over the optimisation path—consistent with $\delta_{\max}$ saturation geometry, where the clip freezes whatever damage has accumulated

Table 6: Ten-cycle continual learning across four models (cycle-10 endpoints). “Plateau from” gives the cycle at which the SLEEP arm’s BCP stops changing. Every SLEEP run shows a step-then-plateau trajectory; the plateau *level* varies by orders of magnitude across seeds of the same configuration.

Model	Seed	SLEEP BCP@10 (plateau from)	Naive BCP@10	SLEEP DRA	Naive DRA
Qwen-7B	0	4.53 (c4)	3.36	0.050	0.085
Qwen-7B	1	10.20 (c3)	6.42	0.032	0.080
Qwen-7B	2	2.71 (c5)	5.44	0.057	0.078
Qwen-1.5B	0	15.96 (c3)	21.01	0.045	0.035
Qwen-1.5B	1	230.1 (c5)	45.83	0.012	0.048
Llama-8B	0	4.83 (c3)	6.09	0.032	0.083
Llama-8B	1	1530.9 (c3)	7.11	0.045	0.075
Mistral-7B	0	3.22 (c2)	2.90	0.008	0.192
Mistral-7B	1	1.64 (c2)	3.72	0.003	0.160

Multi-cycle: SLEEP’s preservation advantage is transient, not architectural

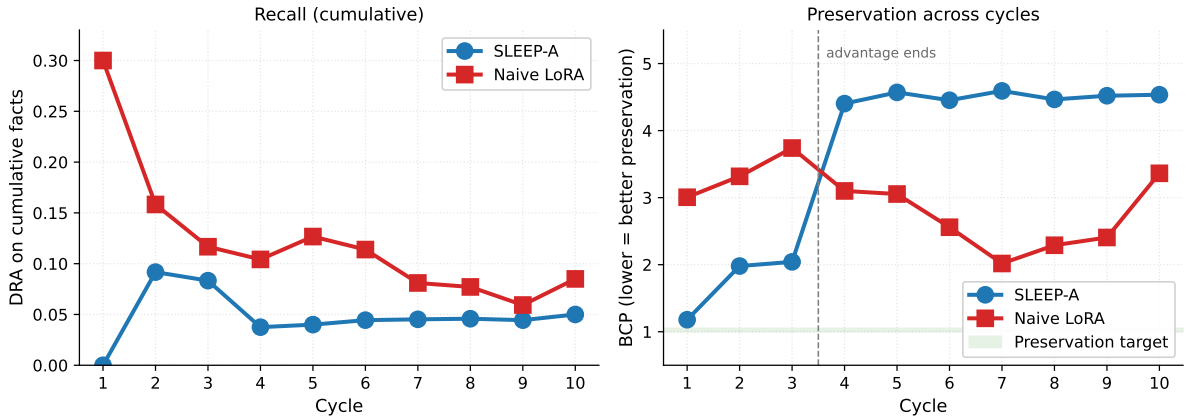


Figure 4: Ten-cycle continual learning on disjoint batches (Qwen2.5-7B, seed 0 shown; Table 6 reports all nine run-pairs). Left: cumulative DRA. Right: BCP with the crossover marked. SLEEP’s preservation advantage holds for three cycles, after which its BCP steps to a plateau while naive LoRA fluctuates.

when saturation is reached—and the unconstrained baseline buys its consistently higher recall with wildly fluctuating damage. We conclude that **single-run, fixed-horizon comparisons of continual-learning architectures are unreliable at both of the scales this field typically reports**, and recommend that such comparisons state a horizon sweep and a seed spread or not be made.

What this result does *not* show: It does not show SLEEP’s safety machinery is useless—in 4 of 9 pairs it produced the better cycle-10 preservation, and its early-cycle protection is real everywhere. It shows the machinery does not deliver a *reliable* advantage, and that the aspirational target—BCP near 1 across many cycles while DRA accumulates—was met by no run of either arm.

7.7 Repairing the Gap: A Retrieval-Aware Warm-Up

Proposed: The recognition–recall diagnosis of Section 7.2 implies a specific repair: teach the frozen model to use KV memory during generation before asking it to consolidate. If the gap is a missing *skill* rather than a structural limit, a lightweight warm-up should narrow it.

Table 7: Retrieval-aware warm-up across four models (two seeds per variant per model; Δ DRA ranges span seeds). Recall moves by at most ± 0.008 anywhere. The gate-only variant is harmless but inert; the +LoRA variant learns its objective yet damages base capability by a family-dependent, sometimes catastrophic amount.

Model	Variant	Δ DRA (seeds)	BCP after (seeds)
Qwen-7B	gate only	+0.002, +0.002	~ 1.00 , ~ 1.00
Qwen-7B	+ LoRA	+0.003, +0.000	1.73, 3.74
Qwen-1.5B	gate only	+0.002, -0.002	1.13, 1.13
Qwen-1.5B	+ LoRA	+0.002, -0.002	5.83, 19.9
Llama-8B	gate only	-0.007, +0.002	1.78, 1.79
Llama-8B	+ LoRA	-0.008, -0.002	1.5×10^5 , 1.5×10^6
Mistral-7B	gate only	+0.008, -0.002	2.77, 3.03
Mistral-7B	+ LoRA	-0.002, -0.005	1.8×10^4 , 1.0×10^5

Retrieval-aware warm-up: recognition improves, recall stays at floor

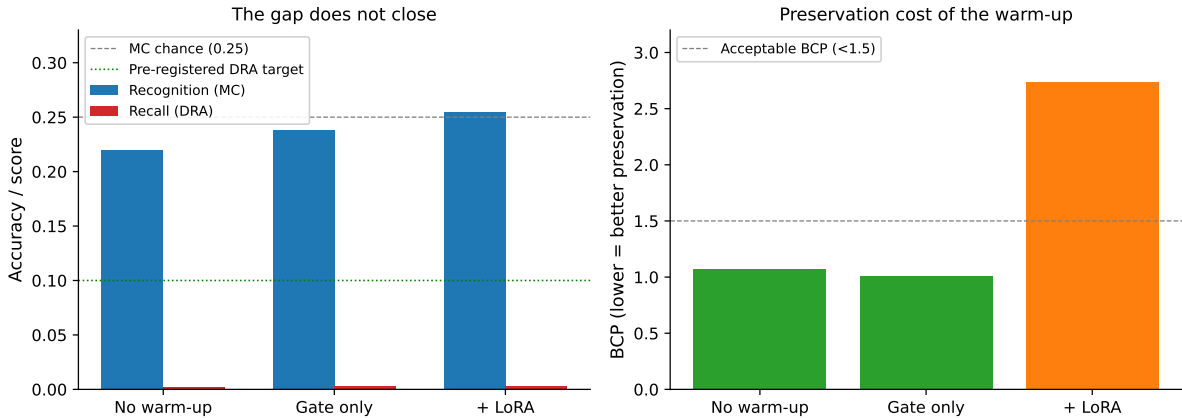


Figure 5: Warm-up extension results. Left: recognition (MC) and recall (DRA) under each condition; the pre-registered DRA target is marked. Recognition rises above chance while recall stays at floor. Right: the preservation cost, showing that the higher-capacity variant buys its small recognition gain with substantial base-capability degradation.

Setup: We run the warm-up of Section 5.6 on 400 WikiText-2 passages disjoint from the evaluation facts, then re-run the identical recognition and recall evaluation on the 200-fact benchmark with the trained component frozen. We test two capacities: the *gate only* (36 parameters) and *gate + LoRA*, in which W_{cons} trains alongside. Success was pre-registered as $\text{DRA} \in [0.10, 0.15]$ at $\text{BCP} < 1.5$.

Result—two distinct failure modes, replicated across families: The gate-only variant does not train: its loss is flat and the learned scales converge to 0.99–1.00, i.e. the optimizer finds that the identity is locally optimal. Thirty-six parameters modulating a memory signal cannot manufacture the capability to use it. The gate + LoRA variant *does* learn the objective—on every model the warm-up loss falls substantially (e.g. 4.30 $\rightarrow$ 3.33 on Qwen-7B), confirming genuine acquisition of memory-conditioned prediction—and recognition improves modestly where the model survives (MC 0.220 $\rightarrow$ 0.270 on Qwen-7B, seed 0). But recall does not follow anywhere: across sixteen warm-up runs on four models, $|\Delta \text{DRA}| \leq 0.008$ (Table 7). Meanwhile the LoRA variant’s preservation cost is family-dependent and sometimes catastrophic: mild on Qwen-7B (BCP 1.7–3.7), severe on Qwen-1.5B (5.8–19.9), and destructive on Llama and Mistral (BCP 10^4 – 10^6) at the same untuned learning rate. Two further observations: when LoRA is present the gate remains pinned at exactly 1.000 (gradient flows to the higher-capacity component), and on Mistral the gate-only

warm-up substantially *reduced* the interference cost of carrying a populated memory bank (BCP with bank active: 8.9 $\rightarrow$ 2.8)—the gate learns to damp memory it cannot learn to exploit.

Interpretation: The honest statement carries its scope. **A lightweight retrieval-aware warm-up— ≤ 300 steps on a general corpus, with either a minimal gate or a rank-16 adapter—does not close the recognition–recall gap on any of four models spanning three families.** What it does do is deepen the original diagnosis in two ways. First, the gap survived a principled intervention drawn from published precedent [Wu et al., 2022, G-MemLLM Authors, 2026, Wang et al., 2024], replicated across families—stronger evidence of its structural character than any single-model observation. Second, the intervention moved recognition and recall *differentially*—the same training that raised MC accuracy left free-form generation untouched—which is precisely the dissociation the paper names, now reproduced under an experimental manipulation rather than merely observed. We note the caveat that the warm-up learning rate was not tuned per family, which the Llama and Mistral collapses underline; a per-family-tuned variant is cheap follow-up work. We do not claim that retrieval-aware training cannot close the gap; Memorizing Transformers demonstrates that full-scale memory-augmented pretraining does. We claim that the cheap version does not, and we report the scale at which we tested it.

8 Discussion

Theoretical contribution: The cleanest empirical signal we measured—a recognition advantage on tagged facts—did not survive transformation into autoregressive generation, and did not survive a deliberate attempt to teach the model to make it survive. We interpret this as a structural property of attention-augmented language models: stored memory K/V provide attention-time logit perturbations that can shift a discrete classification but cannot consistently steer token-by-token sampling against a strong pretrained prior. The warm-up experiment (Section 7.7) turns this from an observation into a manipulation: training that demonstrably taught memory-conditioned prediction raised recognition while leaving recall at floor—on four models across three families—dissociating the two capacities experimentally rather than merely correlationally. Two further conceptual contributions follow. The self-grading result quantifies a failure mode of proxy validation at 93–100% false confirmation on every model tested, which is severe enough to reframe surprise-reduction as unusable in isolation rather than merely imperfect. And the horizon programme shows that a headline comparison between a constrained and an unconstrained architecture is doubly unstable—it inverts with the number of cycles *and* with the random seed, the constrained arm’s damage plateau varying over three orders of magnitude on identical hyperparameters—which bears on how the continual-learning literature reports results generally.

Practical implications: For practitioners, four operational lessons follow. First, direct-write KV memory is usable for *recognition*-style tasks on a frozen backbone—classification, multiple choice, similarity judgement—but not for free-form generation, on any family we tested, and a cheap retrieval-aware warm-up does not change that; closing the gap appears to require memory-augmented training at the scale of Wu et al. [2022]. Second, any system that validates its own learning against a proxy should be audited against an external criterion before that proxy is trusted; on every model, nearly all candidates our proxy certified were rejected by a direct test. Third, continual-learning comparisons should be reported across both a horizon sweep and a seed spread: our nine ten-cycle run-pairs show the winner at a fixed horizon is decided as much by the seed lottery as by the architecture. Fourth, when adopting a compound safety mechanism, ablate it: at 7B scale the EWC-only baseline achieves a better recall-per-preservation ratio than the full pipeline (the effect weakens at 1.5B), which suggests the additional machinery is not carrying its weight in this regime.

Challenges: Several scope constraints bound the strength of our claims:

- (i) We test four models across three families (Qwen2.5-7B/1.5B, Llama-3.1-8B, Mistral-7B), which replicates the recall floor, the self-grading gap, and the warm-up null cross-family—but all are dense decoder-only architectures at ≤ 8 B; mixture-of-experts and much larger scales remain untested.
- (ii) We test *synthetic* facts in 10 templates; natural document content may differ.

- (iii) We test a *single* LoRA configuration (rank 16, V/O projections, top-third layers); other adapter geometries may behave differently.
- (iv) We force *eager attention*; SDPA and FlashAttention paths handle 4D additive masks differently and we did not characterize them after the gating mechanism was finalized.
- (v) The ten-cycle programme runs nine seeded pairs across the four models, which is enough to establish that the plateau level is high-variance but not enough to characterise its *distribution*; a many-seed study of the plateau lottery is open.
- (vi) The warm-up is tested at *one scale* (≤ 300 steps, 400 passages) with a single, untuned learning rate; the Llama/Mistral collapses show this matters. A negative result at this budget does not bound what full memory-augmented pretraining would achieve.

Where variance was measured repeatedly it separates cleanly into two regimes. The single-cycle metrics are tight (DRA SD ≤ 0.003 on every model; the proxy-vs-recall validation gap exceeds any seed fluctuation by two orders of magnitude), so the diagnostic findings are robust. The long-horizon metrics are the opposite—plateau levels vary by up to three orders of magnitude across seeds of one configuration—and we report that variance as a finding rather than averaging over it. The strongest claim our evidence supports is: on these models, on this dataset, with these safety hyperparameters, we observe the patterns reported.

Ethical considerations: Continual-learning systems for language models, if they work, would reduce retraining costs substantially and enable models to learn from interaction. They would also raise privacy and safety concerns: persistent memory of user inputs, model memorization of sensitive information, and asymmetric power between systems that can update themselves and users who cannot. Our specific architecture stores user inputs (tagged spans) in the model’s KV bank and consolidates a subset into permanent weights; deployed naively, this would amount to silent on-device data retention. We argue that any deployed continual-learning system must include explicit user controls over what is encoded, what is consolidated, and what is permanent, alongside mechanisms for selective forgetting on user request. We note additionally that the self-grading result has a safety dimension: a system that certifies its own learning against an internal proxy may report successful consolidation of information it cannot actually reproduce, or fail to report retention of information it can.

9 Conclusion and Future Scope

We implemented and empirically evaluated a biologically-faithful memory consolidation architecture for transformer language models, then attempted to repair the central failure we found, replicating every headline result across four models from three families. Our experiments show that direct-write KV memory injection produces a measurable recognition signal in pretrained transformers but does not transfer to free-form recall—a floor of DRA ≈ 0.006 on every model, from models that follow the same facts in context at up to 0.99; that a retrieval-aware warm-up phase, the repair the diagnosis implies, learns its objective and improves recognition without moving recall in any of sixteen runs; that internal surprise-based validation confirms consolidations at a 93–100% false-confirmation rate against a direct recall test; that no single-cycle configuration achieves useful recall within an acceptable preservation budget; that over nine seeded ten-cycle run-pairs the safety machinery’s damage-capping clip yields a universally-shaped but lottery-levelled preservation plateau, making fixed-horizon single-run comparisons unreliable; and that five specific assumptions in the original formalization required empirical correction before the architecture would function at scale. The key open problem is unchanged and now better specified: bridging from attention-based recognition into weights that support autoregressive generation, without sacrificing base-model preservation.

Future scope: Our results give follow-up work specific targets rather than vague directions:

- (i) *Characterising the plateau lottery:* our nine run-pairs establish that the clip-induced preservation plateau has huge seed variance; a many-seed study could map its distribution, identify what in the optimisation path sets the level, and test whether early trajectory statistics predict it—which would turn the lottery into a controllable quantity.

- (ii) *Retrieval-aware consolidation at scale*: our warm-up failed at ≤ 300 steps with a 36-parameter gate and a rank-16 adapter, on four models. The informative next experiment is the same intervention at the training scale of Memorizing Transformers with per-family-tuned learning rates, which would establish whether the gap is closable in principle within a consolidation pipeline or requires pretraining-time memory augmentation.
- (iii) *External validation criteria*: replace surprise-reduction with the direct recall gate inside the consolidation loop, and measure whether a system that cannot self-certify falsely also consolidates more effectively.
- (iv) *Ablation-first safety design*: given that EWC alone outperformed the compound mechanism at 7B scale, rebuild the safety machinery additively, admitting each component only where it demonstrably improves the recall-per-preservation frontier.
- (v) *Beyond dense $\leq 8B$ decoders*: extend the cross-family matrix to mixture-of-experts architectures and larger scales, where both the recognition signal and the plateau dynamics may differ.

We release the implementation, formalization, and full experimental logs to enable that work.

References

- Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K Dokania, Philip HS Torr, and Marc’Aurelio Ranzato. On tiny episodic memories in continual learning. In *ICML Workshop on Multi-Task and Lifelong Reinforcement Learning*, 2019.
- Susanne Diekelmann and Jan Born. The memory function of sleep. *Nature Reviews Neuroscience*, 11(2): 114–126, 2010.
- Zafeirios Fountas, Martin A Benfeghoul, Adnan Hosseinzadeh, Zhongwei Ma, Jacek Sygnowski, André Barreto, and James Kirkpatrick. Human-like episodic memory for infinite context LLMs. In *International Conference on Learning Representations*, 2025.
- Uwe Frey and Richard GM Morris. Synaptic tagging and long-term potentiation. *Nature*, 385(6616):533–536, 1997.
- G-MemLLM Authors. G-MemLLM: Gated latent memory augmentation for long-context reasoning in LLMs. *arXiv preprint arXiv:2602.00015*, 2026.
- Edward J Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Zhizhong Li and Derek Hoiem. Learning without forgetting. In *European Conference on Computer Vision*, 2017.
- James L McClelland, Bruce L McNaughton, and Randall C O’Reilly. Why there are complementary learning systems in the hippocampus and neocortex. *Psychological Review*, 102(3):419–457, 1995.
- Richard GM Morris. Elements of a neurobiological theory of hippocampal function: the role of synaptic tagging and capture. *European Journal of Neuroscience*, 23(11):2829–2846, 2006.
- Karim Nader, Glenn E Schafe, and Joseph E LeDoux. Fear memories require protein synthesis in the amygdala for reconsolidation after retrieval. *Nature*, 406(6797):722–726, 2000.

- Qwen Team. Qwen2.5 technical report, 2024. <https://qwenlm.github.io/blog/qwen2.5/>.
- Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *CVPR*, 2017.
- Roger L Redondo and Richard GM Morris. Making memories last: the synaptic tagging and capture hypothesis. *Nature Reviews Neuroscience*, 12(1):17–30, 2011.
- Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.
- Hanul Shin, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- Yu Wang, Yifan Gao, Xiushi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, Jingbo Shang, and Julian McAuley. MemoryLLM: Towards self-updatable large language models. In *International Conference on Machine Learning*, 2024.
- Matthew A Wilson and Bruce L McNaughton. Reactivation of hippocampal ensemble memories during sleep. *Science*, 265(5172):676–679, 1994.
- Yuhuai Wu, Markus N Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

Annexure A Formalization Amendments

We resolved 36 design questions in a pre-registered formalization document before any code was written. Empirical work surfaced five amendments, each documented with the experiment that motivated it. We summarize them here; the full formalization (including unchanged questions) is released alongside the implementation.

A.1 ($\delta_{\max}$ must scale with model size): The original specification of $\delta_{\max} = 0.001$ as a per-parameter clip bound saturates within ~ 30 training steps on a 7B model with rank-16 LoRA: 36/36 LoRA parameters hit the clip on every step thereafter, and validation always fails. Required correction: $\delta_{\max} \approx 0.01$ at 7B scale. The per-parameter bound is the wrong abstraction; what matters is the Frobenius norm of the LoRA update relative to the model’s natural weight scale. A scaling rule of $\delta_{\max}(d) = 0.001 \cdot \sqrt{d/768}$ partially corrects this but underestimates the empirically required value.

A.2 (α_{slow} fails for two separable reasons): The original specification of $\alpha_{\text{slow}} = 10^{-5}$ produces gradient updates that round to zero in bfloat16 storage: training loss is completely flat across 100 steps. Bumping to 10^{-4} produces a phase transition (loss $1.46 \rightarrow 0.55$). We subsequently isolated whether this is purely a dtype artefact by re-running the original default in float32 on the same benchmark. It is not. In float32 the loss does move—but only from 2.902 to 2.874 over 100 steps ($\sim 1\%$), with 0/12 validation passes and BCP = 0.998, i.e. the adapter is effectively unmoved. There are therefore *two stacked failures*: a genuine bfloat16 precision floor (updates round to exact zero) *and* a learning rate that is under-scaled for a 7B model in any precision. Required correction: α_{slow} must clear both bounds— $\alpha_{\text{slow}} \geq 10 \cdot \text{ulp}(\text{dtype})$ for representability, and a magnitude that scales with model size for effect. The original 10^{-5} implicitly assumed both float32 storage and a smaller model.

A.3 (μ_{surprise} has a generator-discriminator gap): The quality check “replay must be at least as surprising as the model’s baseline” rejects all replays in practice, because replays are generated from the same model that defines the surprise function and are therefore systematically more in-distribution than external calibration text. The empirical surprise of generated replays (~ 1.25 nats) falls below the baseline computed on calibration text (~ 1.41 nats). Required correction: replace the absolute threshold with a relative-gain criterion ($\text{surprise}(\text{replay}) > \beta \cdot \text{surprise}(\text{seed})$) or compute the threshold from self-generated samples. Until amended, we disable the absolute check.

A.4 (Validation has a data-volume floor): The validation criterion $\text{surprise}_{\text{new}} < (1 - \epsilon_{\text{learn}}) \cdot \text{surprise}_{\text{old}}$ with $\epsilon_{\text{learn}} = 0.10$ is achievable only above a minimum number of replays. With 5 facts and 2–3 replays, 0% of candidates pass; with 200 facts and ~ 50 replays, 40%–60% pass depending on safety settings. The criterion itself is sound, but the formalization should explicitly document that pass rates are zero below a benchmark-dependent volume threshold.

A.5 (Tags are pointers, not storage): The original formalization stored only the K/V projections of tagged tokens, but tagged tokens are short novel-entity mentions (4–8 tokens) that capture the location of the surprise without its content. Required correction: when a tag fires, store K/V for the full sentence containing the tagged span (the *episode*), not just the tagged tokens. This corresponds to the proposal’s own framing of tags as pointers but was implemented as direct storage. Section 7.3 reports the empirical correction.

Annexure B Hyperparameter Table

Annexure C Implementation Details

C.1 KV Memory Injection Mechanics

KV memory injection requires care to avoid silently wrong attention scores. We document the choices made:

Position encoding. We treat memory entries as occupying negative position IDs $[-n_{\text{mem}}, -1]$. RoPE [Su et al., 2024] is applied at read time to memory keys with these positions. The encoding is mathematically valid (sin is odd, cos is even, both well-defined at negative inputs) and verified unit-tested. Memory thus appears to current queries (at positive positions) as “recent context tokens,” a pattern the model was trained on through RoPE’s relative-position mechanic.

Attention mask. Qwen2 uses float additive masks with $-\infty$ for masked positions and 0.0 for visible. When extending the mask with memory columns, we fill those columns with 0.0 of the mask’s own dtype. A boolean True or integer 1 here would silently produce wrong scores on float-mask attention paths.

Cache ordering. Memory K/V must be injected *after* the `past_key_values` cache update, not before. If memory is injected first, the cache accumulates a duplicate copy of memory tokens on every autoregressive generation step; by step 50 the cache contains 50 copies of memory and BCP collapses (we observed $\sim 96\times$ degradation under this bug).

K/V extraction layer-norm. When extracting K/V from a forward pass via `output_hidden_states=True`, the apparent “hidden state at layer ℓ ” is the input to layer ℓ , but attention computes K/V from the layer-normed input. Skipping the layer-norm produces silently wrong K/V. We add an integration test that captures K/V via a forward hook during a normal forward pass and asserts equality with our extraction.

C.2 Per-Phase KV State

A correctness requirement we discovered: KV injection must be *disabled* during W_{cons} training. If left enabled, the model can satisfy the training loss by relying on injected memory rather than encoding the replay content into weights—a crutch the model would not have at inference time after the bank is cleared. We add a `set_enabled(bool)` method to the injector and toggle as follows:

- Wake (tagging, KV write): enabled.

Table 8: Full hyperparameter list. “Source” indicates which of the 36 formalization questions specifies the parameter. “Status” indicates whether the value matches the original specification or was empirically amended.

Parameter	Value	Source	Status
Tagging κ (z-score threshold)	1.5	Q1.2	default
Tagging g (gap tolerance)	3	Q1.3	default
Tagging min span length	4	Q1.3	default
Tagging EMA β	0.99	Q1.2	default
PRP weights (w_E, w_a, w_x, w_r)	(0.35, 0.30, 0.15, 0.20)	Q2.2	default
PRP budget multiplier c_{prp}	500	Q2.4	default
PRP cross-ref similarity threshold	0.5	Q2.2	default
LoRA rank (r)	16	Q3.1	default
LoRA α	32	Q3.1	default
Adapted layer fraction	0.333 (top 1/3)	Q3.2	default
Adapted matrices	V, O	Q3.1	default
α_{fast}	10^{-4}	Q3.3	default (LoRA W_{fast} only)
α_{slow}	10^{-4}	Q3.3	amended (A.2)
δ_{max}	0.01	Q3.4	amended (A.1)
λ_{EWC}	100	Q3.4	default
ϕ_{min} (plasticity floor)	0.10	Q3.4	default
ϵ_{learn} (validation)	0.10	Q4.6	default
$\epsilon_{\text{degrade}}$ (rollback)	0.02	Q4.5	default
Replay temperature	0.7	Q4.1	default
Replay top- p	0.9	Q4.1	default
Compression target	5	Q4.1	default
Steps per memory	5	Q4.4	default
Sleep weight decay	0.01	Q4.4	default
KV bank capacity (tokens)	20,000	—	new (A.5)
KV top- k retrieval gate	16 (default), 64 (episode)	—	new (A.5)
KV memory storage unit	full sentence (episode)	—	amended (A.5)
μ_{surprise} (quality check)	disabled (set to 0)	Q4.1	amended (A.3)

- Phase 1 Generate (replay): enabled (so the model attends to source while compressing).
- Phase 2 Quality check: state preserved (no parameter changes; result invariant).
- Phase 3 Train (W_{cons} updates): *disabled* (forces W_{cons} to encode into weights).
- Phase 4 Perplexity check: disabled (clean preservation measurement).
- Phase 5 Validate & cleanup: disabled (clean recall measurement).
- After successful consolidation: bank is cleared, enabled flag reset for next cycle.

C.3 Memory Gate Placement

The warm-up’s memory gate (Section 5.6) is applied to the memory *values* immediately before they are concatenated onto the key/value time axis, and after the memory keys have received their negative-position RoPE. Placement matters for a reason worth recording: attention output is linear in V but non-linear in K , so gating values scales exactly the memory contribution to the residual stream, whereas gating keys would perturb the softmax allocation itself. A consequence of this choice is that the gate cannot reclaim attention mass—memory positions still absorb softmax weight even at $g \rightarrow 0$ —so it modulates the strength of the memory contribution rather than switching it off. Because the gate is initialised to the identity, installing it is a provable no-op: our first test asserts bit-identical logits against the un-gated path, which caught the alternative placements during development.

Annexure D Multi-Cycle Configuration

The multi-cycle experiment of Section 7.6 uses Setting A from the Pareto sweep ($\delta_{\max} = 0.02$, $\lambda_{\text{EWC}} = 50$, $\alpha_{\text{slow}} = 10^{-4}$, steps-per-memory = 4) for the SLEEP arm. The naive LoRA arm uses 200 gradient steps per cycle on the cycle’s batch only, with AdamW (lr 10^{-4} , weight decay 0.01, batch size 32, uniform-with-replacement sampling). Both arms accumulate adapter state across cycles; weights from cycle i persist into cycle $i + 1$. Both arms are evaluated on identical control texts and recall benchmarks after each cycle. Replay strategy for the SLEEP arm is set to **original** (replays are the original fact tokens), eliminating replay quality as a confound and isolating the multi-cycle dynamics of the consolidation training step. The ten-cycle programme divides the 200 facts into 10 disjoint batches of 20 and repeats the paired protocol with three seeds on Qwen2.5-7B and two seeds on each of Qwen2.5-1.5B, Llama-3.1-8B, and Mistral-7B (nine run-pairs). The earlier three-cycle configuration used batches of ≈ 67 , and we note that batch size per cycle is a candidate moderator of the plateau onset.

Annexure E Reproducibility Notes

Seeding. A single entry point seeds Python’s **random**, NumPy, PyTorch (CPU and CUDA), and PYTHONHASHSEED. Multi-seed experiments are executed one seed per process so that model initialisation is genuinely independent rather than sequentially derived; the aggregating harness collects per-run JSON and reports mean and sample standard deviation.

Environment sensitivity. Three environment issues materially affected results and are recorded so that replication does not rediscover them. First, **transformers** must be pinned to the version documented in the repository: a later release imports a distributed-tensor symbol unavailable in the CUDA image’s PyTorch build, and fails at import. Second, adapter modules must be constructed in the wrapped layer’s dtype; a float32 adapter attached to a bfloat16 model raises a dtype mismatch at the first matrix multiply. Third, the patched attention forward must not assume architecture-specific attributes: **sliding_window** exists on Qwen2 attention modules but not on Llama’s or (in the pinned release) Mistral’s, and is read defensively. The cross-architecture verification harness of Section 6 caught the third issue before any invalid run executed; all are recorded in the released logs.

Verification practice. Every modification to the KV-injection path is accompanied by a hook-based equality test against the model’s own computation, written before the feature it verifies. This practice caught two silent-failure bugs during development: a missing layer-norm in K/V extraction, and a cache-ordering error that duplicated memory on every generation step. The memory gate’s identity-initialisation test follows the same pattern.